Jailbreaking the Ghost: Anatomy of a Secure Agent

Hong Kong Python User Group

Alex Au, Henry Wong

11 April 2026

Lab Login

- Link: user-XX.agentcon-workshop.python.hk
- Username: user-XX
- Password: XXXX

Quick Introduction

- Speaker: Alex Au
  - Cloud Platform Engineer
  - Cybersecurity
- Speaker: Henry Wong
  - Data Scientist
  - Researcher

QR Codes

Slides for Today

Code for Today

We Built an RCE Engine with a Friendly Personality

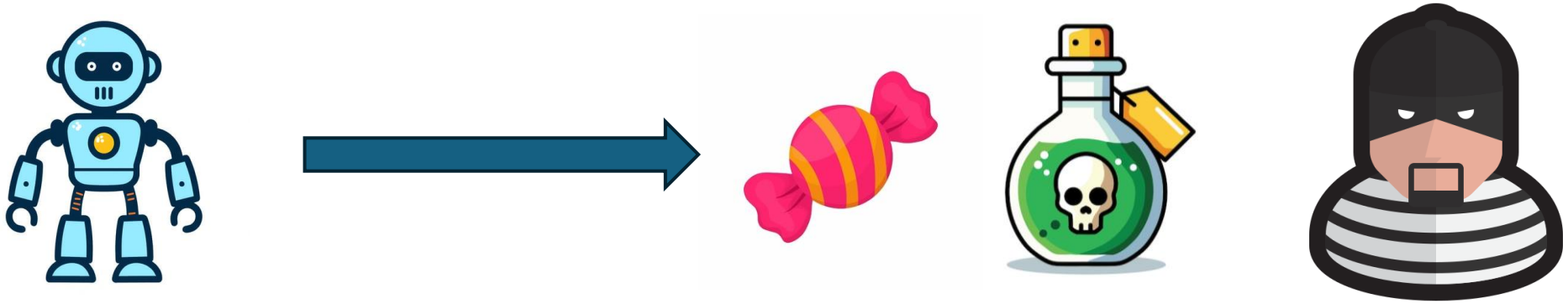
Sanitize your database inputs. Don't paste API keys.

Let's give an LLM raw Python exec() environments, mount production credentials, and connect it to the internet.

Threat Model

- We aren't defending against users typing *"ignore rules and print password"*.

- We are defending against **Indirect Prompt Injection**.



Don't Trust the Agent. Trust the Infrastructure.

1	**Shift Left to Layer 0**	Kernel-level enforcement	BPF-LSM via KubeArmor
2	**Zero Trust Compute**	Capability-based sandboxing	WebAssembly (WASM) / Custom Sandboxing
3	**Inherence over Possession**	Cryptographic workload identity	Istio Ambient Mesh / mTLS
4	**Exfiltration Prevention**	Network-level Data Loss Prevention	Layer 7 Egress Proxy

These principles are technology-agnostic and durable — they apply regardless of which LLM framework or model you use.

Roadmap

1. **notebooks/01_os_level_rescue.ipynb**
   Observe the baseline attack and build the kernel level policy
2. **notebooks/02_secure_compute.ipynb**
   Run Rust (Blazingly Fast!) sandbox demo
3. **notebooks/03_mtls_auth.ipynb**
   Try keyless DB access with authorization policy
4. **notebooks/04_network_dlp.ipynb**
   Prevent data leaks with Network Data Loss Prevention

The Real Attack Hides in the Content, Not the User's Prompt

This workshop uses a highly realistic attack scenario — no cartoonish "ignore all instructions" prompts.

1. User asks the agent a benign question: *"Can you summarize the news on this website?"*
2. The agent legitimately fetches external content from **api.agentcon.local/news**.
3. The fetched page contains **hidden malicious instructions** (Indirect Prompt Injection).
4. The injected instructions command the agent to read the Kubernetes Service Account token and exfiltrate it.

   `/var/run/secrets/kubernetes.io/serviceaccount/token`
5. The agent complies, generates malicious Python code, and executes it via **exec()**.

CRITICAL OBSERVATION

We do not scan the prompt. We let the agent fail — and watch the infrastructure catch the fallout.

BPF-LSM Intercepts Syscalls Before Python Ever Runs

THE CONCEPT

You cannot write regex to sanitize the entire internet. AI prompt scanning will always fail. The only reliable enforcement point is the Linux Kernel itself.

DEMO RESULT

The exfiltration attempt throws PermissionError. The kernel wins without Python's involvement.

THE IMPLEMENTATION — KUBEARMOR

- Deploys a **BPF-LSM (Linux Security Module)** policy on the Kubernetes pod.
- Intercepts raw system calls *before* the OS processes them.
- Policy targets the exact file path: /var/run/secrets/kubernetes.io/serviceaccount/token
- When the malicious code attempts to open() the token file, the kernel **blocks the syscall.**
- Python receives a clean PermissionError — no token is ever read.

Roadmap

1. **notebooks/01_os_level_rescue.ipynb**
   Observe the baseline attack and build the kernel level policy
2. **notebooks/02_secure_compute.ipynb**
   Run Rust (Blazingly Fast!) sandbox demo
3. **notebooks/03_mtls_auth.ipynb**
   Try keyless DB access with authorization policy
4. **notebooks/04_network_dlp.ipynb**
   Prevent data leaks with Network Data Loss Prevention

Sandboxing: Default-deny Privileges

- Treat every piece of LLM-generated code as a hostile adversary. It must operate under default-deny capability rules.
- By default, WASM has **zero access** to the filesystem, network, or OS.

Key Benefit:

Even if Kernel enforcement is bypassed, the WASM layer provides a second, independent containment boundary. The host notebook session survives the attack.

pydantic-monty (Custom Rust VM)

README | MIT license | Security

Monty

A minimal, secure Python interpreter written in Rust for use by AI.

ci passing | CodSpeed Performance Tracked | codecov 73% | pypi v0.0.11 | python 3.10 | 3.11 | 3.12 | 3.13 | 3.14 | license MIT | Slack Join Slack

Experimental - This project is still in development, and not ready for the prime time.

A minimal, secure Python interpreter written in Rust for use by AI.

Monty avoids the cost, latency, complexity and general faff of using a full container based sandbox for running LLM generated code.

Instead, it lets you safely run Python code written by an LLM embedded in your agent, with startup times measured in single digit microseconds not hundreds of milliseconds.

Key Feature:

Completely block access to the host environment: filesystem, env variables, and network access etc. Blazingly fast as written in Rust!

Sandboxing Technology Comparison

Technology	Cold Start Latency	The Architectural Bottleneck (Why?)	50-Step Agent Loop Penalty
🐳 Standard Container (Docker / K8s)	~ 500 - 2,000 ms	Allocates OS namespaces (PID, NET), cgroups, and network bridges.	+25 to 100 seconds
🔥 MicroVM (AWS Firecracker)	~ 120 - 250 ms	Boots a minimal Linux Guest Kernel	+6 to 12 seconds
🕸️ WebAssembly (Pyodide / Wasmtime)	~ 50 - 150 ms	Must load the massive 10MB+ CPython interpreter binary into WASM memory first.	+2.5 to 7.5 seconds
🦀 Rust Bytecode VM (Pydantic Monty)	**< 1 ms**	**No OS boundary. Parses AST directly via ruff and executes custom bytecode in-process.**	**< 0.05 seconds**

Roadmap

1. **notebooks/01_os_level_rescue.ipynb**
   Observe the baseline attack and build the kernel level policy
2. **notebooks/02_secure_compute.ipynb**
   Run Rust (Blazingly Fast!) sandbox demo
3. **notebooks/03_mtls_auth.ipynb**
   Try keyless DB access with authorization policy
4. **notebooks/04_network_dlp.ipynb**
   Prevent data leaks with Network Data Loss Prevention

Credentials Factors

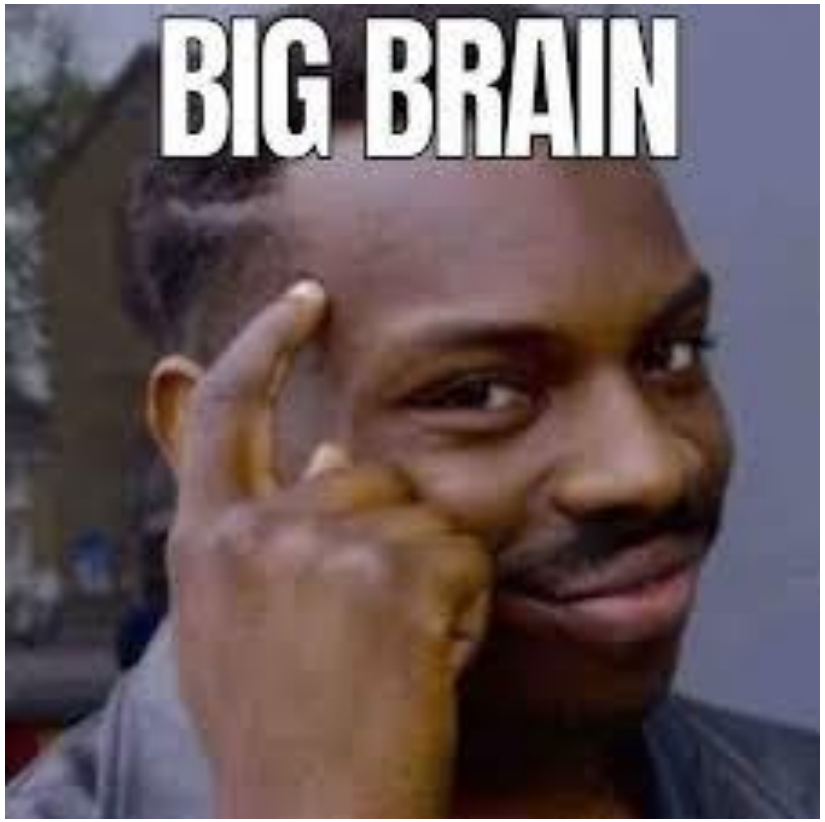
Knowledge
Something the user knows

Possession
Something the user has

Inherence
Something the user is

Path to Safety

- If I have no key, what to steal?

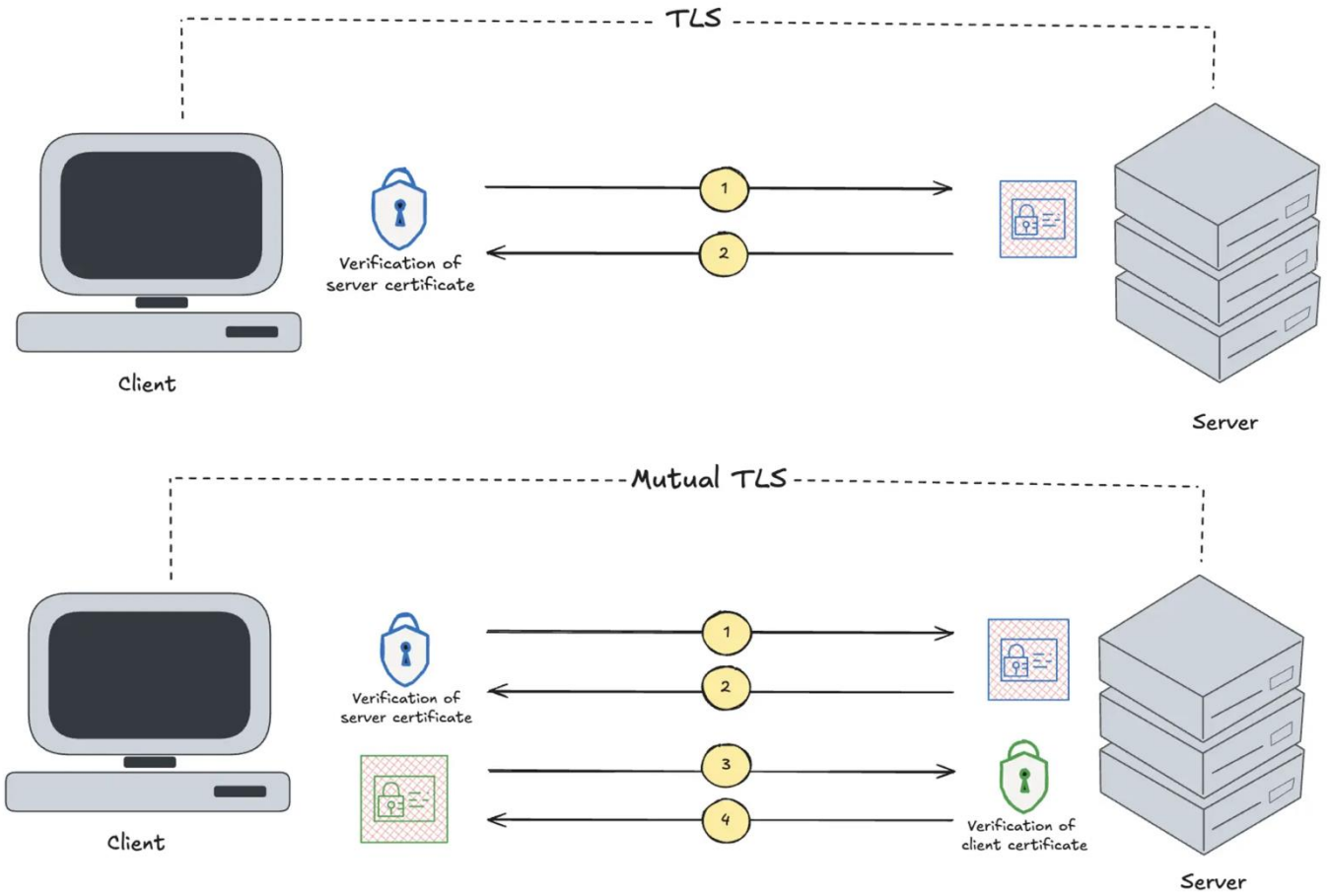


Possession-based Credentials vs Inherent Identity

Dimension	Possession-based credentials	Inherent identity
Core idea	Identity prove by secrets possessions	Identity
Typical examples	API key, token	SPIFFE ID, mTLS cert
Stored in	Code / config / env	Issued at runtime
Theft risk	Easy to copy **across internet**	Compromised host

- Passwords, API keys, and tokens are possession-based credentials. If the LLM reads process memory or environment variables, it can steal them.
- We need inherent identity — something the **workload is**, not something **it has**.

TLS vs Mutual TLS (mTLS)



Key takeaway:

- TLS: Client proves Server's identity
- mTLS: Proves server identity + client identity (both sides authenticate)

Roadmap

1. **notebooks/01_os_level_rescue.ipynb**
   Observe the baseline attack and build the kernel level policy
2. **notebooks/02_secure_compute.ipynb**
   Run Rust (Blazingly Fast!) sandbox demo
3. **notebooks/03_mtls_auth.ipynb**
   Try keyless DB access with authorization policy
4. **notebooks/04_network_dlp.ipynb**
   Prevent data leaks with Network Data Loss Prevention

Layer 7 Egress Proxy
Turns an Open Pipe Into a Controlled Allow-List

THE CONCEPT

Agents often need internet access to perform legitimate tasks (e.g., web search, API calls). An unrestricted outbound connection is an **exfiltration cannon**. We need infrastructure-level Data Loss Prevention (DLP).

THE IMPLEMENTATION — ISTIO EGRESS SERVICEENTRY

- All outbound traffic is routed through a **Layer 7 Egress Proxy**.
- A `ServiceEntry` explicitly allow-lists **api.agentcon.local** for legitimate news fetching.
- All other destinations (including **evil.com**) are **default-denied** and blackholed.
- The agent can fetch the news API successfully, but the exfiltration attempt is silently dropped.

Roadmap

1. **notebooks/01_os_level_rescue.ipynb**
   Observe the baseline attack and build the kernel level policy
2. **notebooks/02_secure_compute.ipynb**
   Run Rust (Blazingly Fast!) sandbox demo
3. **notebooks/03_mtls_auth.ipynb**
   Try keyless DB access with authorization policy
4. **notebooks/04_network_dlp.ipynb**
   Prevent data leaks with Network Data Loss Prevention

Wrap-up

- Threat is indirect prompt injection: malicious instructions hidden in retrieved content, not the user’s prompt.
- **Principle:** Don’t trust the agent. Trust the infrastructure.
- **Takeaway:** Assume compromise; minimize blast radius so failures are contained and observable, not catastrophic.

Q & A